

PAKHI'S COLLECTION

Overall E-Commerce Project Blueprint

ELEGANCE IN EVERY THREAD

A build-ready specification for creating a modern Indian women's ethnic-fashion store with AI coding tools. Launch collection: Sarees and Women's Kurtas. Architecture is designed to expand into additional fashion categories later.

Project	Pakhi's Collection
Launch products	Sarees + Women's Kurtas
Market	India
Currency	INR (₹)
Payments	UPI + Debit/Credit Cards + Cash on Delivery
Core platform	Responsive web e-commerce
Admin	Full product, inventory, order and analytics dashboard
Future	Dresses, Lehengas, Kurta Sets, Suits, Dupattas, Jewellery, Accessories, etc.

Recommended brand direction: premium, elegant, minimal Indian ethnic fashion.

1. Project Vision

Pakhi's Collection should launch as a focused Saree and Women's Kurta store rather than a generic fashion marketplace. The backend must remain category-driven so future collections can be added without rebuilding the application.

- Fast, mobile-first shopping experience.
- Premium visual identity with clean product photography.
- Reliable checkout with UPI, cards and COD.
- Admin-controlled catalog, inventory, orders, coupons and returns.
- AI-assisted search, recommendations and size guidance as later phases.
- Secure server-side payment verification and role-based administration.

2. Brand & UI Direction

Element	Recommendation
Brand	Pakhi's Collection
Tagline	Elegance in Every Thread
Primary	#722F3D — wine/burgundy
Background	#F5EBDD — warm ivory
Accent	#C6A36B — muted gold
Text	#302522 — espresso
Style	Premium, elegant, minimal, Indian ethnic
Mobile priority	Yes

3. Launch Catalog

Sarees

- Cotton
- Silk
- Georgette
- Chiffon
- Organza
- Banarasi
- Printed
- Embroidered
- Party/Wedding
- Handloom

Women's Kurtas

- Straight
- Anarkali
- A-Line
- Printed
- Embroidered
- Cotton
- Rayon
- Festive
- Casual
- Office wear

Future categories

Dresses, Lehengas, Kurta Sets, Salwar Suits, Dupattas, Jewellery, Bags, Footwear and other accessories.

4. Customer Website Pages

Route / Area	Purpose
/	Homepage, hero, collections, new arrivals, trending, festive edits
/shop	All products with sorting and filtering
/category/[slug]	Category-specific catalog
/product/[slug]	Product gallery, variants, details, reviews, recommendations
/search	Keyword and natural-language search
/wishlist	Saved products
/cart	Cart and coupon
/checkout	Address, shipping, payment, order confirmation
/account	Profile, orders, addresses, wishlist, returns
/orders/[id]	Order status and tracking
/returns	Return/refund requests
/about, /contact, /faq	Business information and support
/policies/*	Privacy, terms, shipping, returns

5. Homepage Blueprint

NAVBAR

Logo | Shop | Sarees | Kurtas | New Arrivals | Search | Wishlist | Account | Cart

HERO

"Elegance in Every Thread"

[SHOP SAREES] [SHOP KURTAS]

SHOP BY CATEGORY

Sarees | Women's Kurtas

NEW ARRIVALS
Product cards

TRENDING SAREES
Product carousel

KURTA COLLECTION
Product carousel

SHOP BY OCCASION
Wedding | Festive | Office | Casual | Party

SHOP BY FABRIC / STYLE
Silk | Cotton | Georgette | Organza | Anarkali | A-Line | Printed

COMPLETE THE LOOK
Recommended combinations

TRUST SECTION
Secure payments | Easy returns | Order tracking | Customer support

FOOTER
Policies | Contact | Social links | Newsletter

6. Product Model

Common product fields

- Name
- Slug
- Description
- Category
- Subcategory
- Brand
- Base price
- Sale price
- SKU
- Images
- Fabric
- Pattern
- Occasion
- Status
- Tags
- SEO metadata

Saree-specific attributes

- Saree length
- Blouse length
- Blouse included
- Blouse color
- Work/embellishment

- Fabric
- Pattern
- Occasion
- Care instructions

Kurta-specific attributes

- Sizes
- Color
- Fabric
- Pattern
- Fit
- Sleeve
- Neck
- Length
- Occasion
- Care instructions

Use flexible product attributes or a structured attribute system. Do not hard-code Saree/Kurta logic throughout the application.

7. Search, Filters & Discovery

- Keyword search with typo tolerance.
- Natural-language search such as: 'red saree for wedding under ₹3000'.
- Saree filters: price, color, fabric, pattern, occasion, work, blouse included, rating, discount.
- Kurta filters: price, size, color, fabric, pattern, fit, sleeve, neck, occasion, rating, discount.
- Sorting: popular, newest, price low-high, price high-low, rating, discount.
- Recently viewed products.
- Related products and 'You may also like'.

8. Cart & Checkout

CART

Product → Variant → Quantity → Price → Discount → Remove/Save for later

CHECKOUT

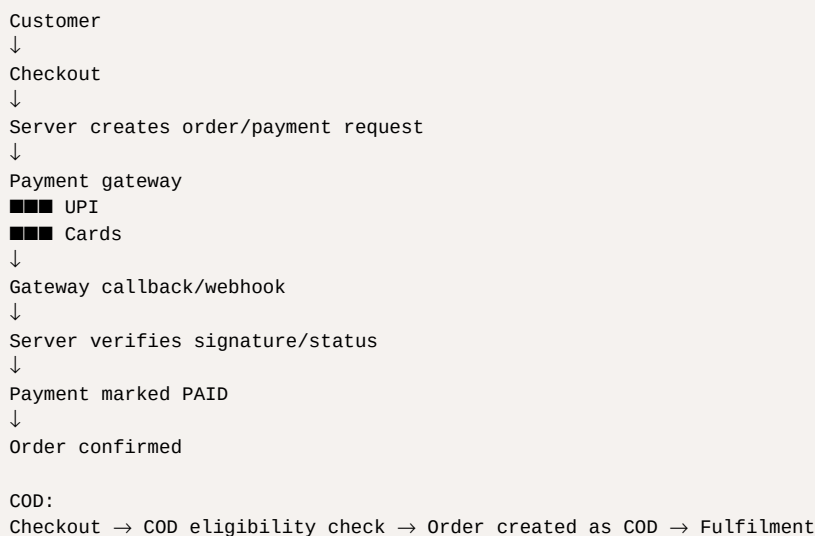
1. Customer login/guest policy
2. Delivery address
3. Shipping method
4. Coupon
5. Payment method
6. Order summary
7. Place order
8. Confirmation + tracking

PAYMENT OPTIONS

- UPI
- Debit/Credit Card
- Cash on Delivery

The server must calculate the final price from database values. Never trust a price, discount, stock count or total sent by the browser.

9. Payment Architecture



Use a production payment gateway such as Razorpay or Cashfree. Keep gateway secret keys server-side. Use test/sandbox mode during development. Implement webhook signature verification and idempotency so duplicate callbacks cannot create duplicate orders.

10. COD Rules

- Configurable maximum COD order value.
- Optional PIN/serviceability check.
- Optional small prepaid amount for high-risk/high-value COD orders.
- Record COD status separately from online payment status.
- Admin can disable COD globally or by product/category/region if required.

11. Order Lifecycle



Every status change should be recorded in an order-status history table for auditability.

12. Inventory

```
PRODUCT
■■■■ VARIANTS
■■■■ SKU
■■■■ Size
■■■■ Color
■■■■ Stock
```

Kurta:

S → 12

M → 18

L → 14

XL → 7

Saree:

One-size variant → 14

- Stock is variant-level, not just product-level.
- Prevent checkout when stock is insufficient.
- Reserve/decrement stock safely during order processing.
- Show out-of-stock state.
- Low-stock threshold and admin alerts.
- Maintain inventory transaction history.

13. Returns & Refunds

```
Customer
→ Select order item
→ Select reason
→ Submit return request
```

```
Admin
→ Review
→ Approve/Reject
→ Pickup/Receive
→ Quality check
→ Refund
→ Close request
```

Return policy, eligible products, time windows, refund method and conditions must be configured according to the business's actual policy and applicable Indian consumer/e-commerce requirements.

14. Customer Account

- Phone OTP and/or email authentication.
- Profile.
- Multiple delivery addresses.
- Order history.
- Wishlist.
- Saved size information.
- Return/refund history.
- Notifications.
- Logout and account security controls.

15. Admin Dashboard

Module	Functions
Dashboard	Revenue, orders, customers, products, alerts
Products	Create, edit, archive, images, SEO, variants
Categories	Create/reorder categories and subcategories
Inventory	Stock, adjustments, low-stock alerts, inventory history
Orders	View, filter, update status, invoices, fulfilment
Payments	Payment status, gateway IDs, refunds
Customers	Profiles, order history, support notes
Coupons	Codes, percentage/fixed discounts, limits, expiry
Returns	Review, approve/reject, refund workflow
Reviews	Moderation and verified-purchase checks
Banners	Homepage promotional content
Analytics	Revenue, AOV, conversion, products, payment mix
Settings	Store, shipping, COD, tax, notification settings

16. Database Blueprint

```
users
- id
- name
- email
- phone
- password_hash (if password auth)
- role
- created_at
- updated_at

categories
- id
- parent_id
- name
- slug
- image
- sort_order
- status

products
- id
- category_id
- name
- slug
- description
- base_price
- sale_price
- fabric
- pattern
- fit
- occasion
- status
- created_at
- updated_at
```

```
product_variants
- id
- product_id
- sku
- size
- color
- price
- stock

product_images
- id
- product_id
- image_url
- position

carts
- id
- user_id

cart_items
- id
- cart_id
- product_variant_id
- quantity

wishlists
- id
- user_id

wishlist_items
- id
- wishlist_id
- product_id

addresses
- id
- user_id
- name
- phone
- address
- city
- state
- pincode

orders
- id
- order_number
- user_id
- subtotal
- discount
- shipping_fee
- tax
- total
- payment_method
- payment_status
- order_status
- address_snapshot
- created_at
- updated_at

order_items
- id
- order_id
- product_variant_id
- product_name_snapshot
- variant_snapshot
- price_snapshot
- quantity

payments
- id
- order_id
```

- gateway
- gateway_order_id
- gateway_payment_id
- amount
- currency
- status
- method

coupons

- id
- code
- type
- value
- minimum_order
- maximum_discount
- usage_limit
- used_count
- starts_at
- expires_at

reviews

- id
- user_id
- product_id
- order_item_id
- rating
- title
- comment
- verified_purchase

returns

- id
- order_item_id
- user_id
- reason
- status
- refund_amount

notifications

- id
- user_id
- type
- title
- message
- read
- created_at

order_status_history

- id
- order_id
- old_status
- new_status
- changed_by
- created_at

inventory_transactions

- id
- variant_id
- type
- quantity
- reference
- created_at

17. Recommended Tech Stack

Layer	Technology
Frontend	Next.js + React + TypeScript

Layer	Technology
UI	Tailwind CSS + reusable component system
Backend	Next.js server routes/actions or Node.js service
Database	PostgreSQL
ORM	Prisma
Authentication	Auth.js and/or OTP provider
Payments	Razorpay or Cashfree
Images	Cloudinary or equivalent object storage/CDN
Email	Transactional email provider
SMS/WhatsApp	Approved transactional provider
Hosting	Vercel or equivalent
Version control	GitHub
Analytics	Google Analytics + internal admin analytics

18. Project Folder Structure

```

pakhis-collection/
├── app/
│   ├── page.tsx
│   ├── shop/
│   ├── category/[slug]/
│   ├── product/[slug]/
│   ├── search/
│   ├── wishlist/
│   ├── cart/
│   ├── checkout/
│   ├── account/
│   ├── orders/[id]/
│   ├── admin/
│   │   ├── dashboard/
│   │   ├── products/
│   │   ├── categories/
│   │   ├── inventory/
│   │   ├── orders/
│   │   ├── customers/
│   │   ├── coupons/
│   │   ├── returns/
│   │   ├── analytics/
│   │   ├── api/
│   │   ├── products/
│   │   ├── cart/
│   │   ├── orders/
│   │   ├── payments/
│   │   ├── webhooks/
│   │   ├── auth/
│   │   ├── components/
│   │   ├── lib/
│   │   ├── db.ts
│   │   ├── auth.ts
│   │   ├── payments.ts
│   │   ├── pricing.ts
│   │   ├── inventory.ts
│   │   ├── notifications.ts
│   │   ├── prisma/
│   │   ├── schema.prisma
│   │   ├── public/
│   │   ├── tests/

```

```
■■■ .env.example
■■■ package.json
■■■ README.md
```

19. Security Requirements

- HTTPS in production.
- Secure authentication and password hashing if passwords are used.
- Role-based authorization for admin routes.
- Server-side validation of all important inputs.
- Never trust client-provided prices, discounts, stock or payment status.
- Payment webhook signature verification.
- Rate limiting on authentication, checkout and sensitive endpoints.
- Secure cookies and session handling.
- Upload type/size validation for images.
- Audit logs for sensitive admin actions.
- Database backups.
- Secrets only in environment variables or a secrets manager.
- Do not store raw card numbers, CVV or other sensitive card data.

20. AI Features — Phase 2

- Natural-language product search.
- Personalized product recommendations.
- Complete-the-look suggestions.
- Size/fit assistant based on the store's real size charts and product measurements.
- AI-generated draft product descriptions for admin review.
- Customer support assistant for store policies and product questions.
- Abandoned-cart messaging, subject to the chosen messaging provider and applicable consent/marketing rules.

AI should assist the store; it should not invent product attributes, stock, pricing, discounts, delivery promises or return-policy rules.

21. Notifications

- Order confirmation.
- Payment confirmation/failure.
- Order shipped.
- Out for delivery.
- Delivered.
- Return request updates.
- Refund updates.

- Optional abandoned-cart and promotional messaging.

22. SEO & Marketing

- Unique product/category titles and meta descriptions.
- SEO-friendly slugs.
- Product structured data where appropriate.
- Optimized image sizes and alt text.
- Sitemap and robots configuration.
- Canonical URLs.
- Social sharing metadata.
- Google Analytics and Search Console.
- Campaign/UTM tracking.

23. Testing Checklist

Area	Test cases
Auth	Register, OTP, login, logout, session expiry, role access
Catalog	Search, filters, sorting, product variants, out-of-stock
Cart	Add/remove/update quantity, price recalculation, stock validation
Coupons	Valid, invalid, expired, limit, minimum order, max discount
Payments	UPI/card success, failure, cancellation, webhook, duplicate webhook
COD	Eligible/ineligible orders, limits, status
Orders	Create, confirm, ship, deliver, cancel, return, refund
Admin	CRUD, authorization, inventory, analytics
Security	Input validation, rate limits, secret exposure, authorization
Mobile	Responsive layouts, checkout usability, navigation
Performance	Images, page load, database queries, caching

24. Development Roadmap

Phase	Deliverables
0 — Planning	Brand, requirements, UI wireframes, schema, architecture
1 — Foundation	Next.js, TypeScript, Tailwind, PostgreSQL, Prisma, environments
2 — Catalog	Categories, products, variants, images, search, filters
3 — Accounts	Auth, profiles, addresses, wishlist
4 — Commerce	Cart, coupons, checkout, order creation
5 — Payments	UPI/cards gateway, webhooks, payment verification, COD

Phase	Deliverables
6 — Admin	Products, inventory, orders, customers, coupons, returns
7 — Shipping	Courier/shipping integration, tracking, notifications
8 — Quality	Testing, security, performance, SEO, accessibility
9 — Launch	Production deployment, domain, monitoring, backups
10 — AI	Search, recommendations, size assistant, support assistant

25. AI Coding Strategy

Do not ask an AI coding tool to generate the entire production store in one prompt. Build it in controlled milestones so the database, business rules and payment security remain understandable and testable.

Prompt 1 — Project foundation

You are the lead full-stack engineer for Pakhi's Collection, an Indian women's ethnic fashion e-commerce store. Build a production-ready Next.js + TypeScript + Tailwind project with PostgreSQL + Prisma. The launch catalog is Sarees and Women's Kurtas, but the catalog must be category-driven and expandable. Set up clean architecture, environment variables, linting, formatting, reusable components and a README. Do not implement payments yet. First create the foundation and explain every major file.

Prompt 2 — Database

Implement the PostgreSQL/Prisma schema for Pakhi's Collection using the blueprint. Include users, roles, categories, products, variants, images, carts, wishlist, addresses, orders, order items, payments, coupons, reviews, returns, notifications, order status history and inventory transactions. Add indexes and unique constraints where appropriate. Create safe migrations and seed data for Sarees and Women's Kurtas.

Prompt 3 — Storefront

Build the customer storefront from the blueprint: responsive navbar, homepage, category pages, product listing, product detail, search, filters, sorting, wishlist and cart. Use database-driven categories and variants. Make it mobile-first and premium Indian ethnic fashion themed. Do not use fake product data once the database is connected.

Prompt 4 — Checkout

Implement secure cart and checkout. The server must calculate product prices, discounts, taxes and shipping from trusted database values. Validate stock server-side. Create an order record before payment where appropriate. Support UPI/card gateway integration through a payment abstraction and Cash on Delivery. Keep all secrets server-side. Add clear error states and idempotency protections.

Prompt 5 — Admin

Build the /admin dashboard with role-based access. Add product/category CRUD, image management, variant/stock management, orders, customers, coupons, returns, reviews, banners and analytics. Every sensitive admin action must be authorized server-side and important changes should be auditable.

Prompt 6 — Testing & security

Audit the entire Pakhi's Collection application for security, correctness and production readiness. Test authentication, authorization, cart pricing, stock, coupons, payment webhooks, duplicate callbacks, COD, returns, admin access, input validation, rate limiting and secret exposure. Fix issues rather than merely listing them. Add automated tests for critical business logic.

Prompt 7 — AI features

Add AI features only after the core store is stable. Implement natural-language product search, recommendations, complete-the-look and a size assistant using real product attributes and size charts. AI must never invent stock, prices, product specifications or policy information. Add fallback behavior when the AI service is unavailable.

26. Launch Checklist

- Domain connected.
- Production database configured and backed up.
- Payment gateway account verified and production credentials installed securely.
- UPI/card test transactions completed.
- COD rules configured.
- Shipping/serviceability configured.
- Return/refund policy published.
- Privacy policy and terms published.
- Product photos optimized.
- Product stock checked.
- Admin accounts secured.
- Analytics installed.
- Error monitoring enabled.
- Mobile checkout tested.
- Backup and recovery procedure tested.
- Final security review completed.

27. Final Architecture

```
PAKHI'S COLLECTION
|
+-----+-----+
| | |
CUSTOMER ADMIN SERVICES
| | |
Next.js Admin UI Email/SMS/WhatsApp
| |
+----- API / SERVER -----+
|
+-----+-----+
| | |
PostgreSQL Payment Gateway Image CDN
| | |
Products/Users UPI + Cards Product Media
Orders/Inventory
|
Order System
|
Shipping
|
Customer
```

28. Product Expansion Strategy

When Pakhi's Collection expands, add categories through the admin/catalog system rather than changing the fundamental architecture.

```
Launch:
Sarees
Women's Kurtas

Later:
```

Kurta Sets
Lehengas
Salwar Suits
Dresses
Dupattas
Jewellery
Bags
Footwear
Accessories

The same:
catalog → cart → checkout → payment → order → inventory → shipping → reviews
pipeline should support every new category.

End of blueprint — use this document as the master specification when building Pakhi's Collection with AI coding tools.